SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME: Data Structures **COURSE CODE: CSA0308**

Register Number	**192411185**
Name	**CH.BHARATH KUMAR REDDY**

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions:4 **EACH QUESTIONS CARRIES:** 25 MARKS **Total Marks:**100 **Duration:** ONE HOUR

S. No.	List of Questions
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
3	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
4	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and Prepare a technical presentation justifying using time complexity comparison.

Code of Conduct:

I **CH. BHARATH KUMAR REDDY (*192411185*)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

CH.BHARATH KUMAR REDDY
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

1 ANS

Student Management System – Primitive and Non-Primitive Data Structures

Introduction

A student management system stores and manages the marks of 1000 students. A suitable data structure is required to store, access, and process student marks efficiently.

Data Structures Used

Primitive Data Structures:

- int – stores student marks.
- char – stores grade or other character data.

Non-Primitive Data Structures:

- Array – stores marks of multiple students.
- Structure (struct) – stores multiple details of each student, such as roll number, name, and marks.

Proposed Solution

Use an array of structures to store student details.

```
#include <stdio.h>

struct Student {
    int rollNo;
    char name[30];
    int marks;
};

int main() {
    struct Student s[1000];

    s[0].rollNo = 101;
    s[0].marks = 90;

    printf("Roll Number: %d\n", s[0].rollNo);
    printf("Marks: %d\n", s[0].marks);

    return 0;
}
```

Output:

Roll Number: 101

Marks: 90

Technical Justification

- An array stores 1000 student records in indexed positions.
- A structure combines related information into one record.
- Marks can be accessed directly using an index.
- Sequential traversal can calculate total marks and average marks.
- The solution is simple and memory-efficient when the number of students is fixed.

Time and Space Complexity

Operation	Complexity
Access by index	\(O(1)\)
Search by roll number	\(O(n)\)
Traverse all students	\(O(n)\)
Storage	\(O(n)\)

Here, (n = 1000).

Conclusion

An array of structures is suitable because it stores student records in an organized manner and provides fast indexed access. If frequent insertion and deletion are required, a linked list or database-backed structure may be more appropriate.

2.ANS.

Navigation System – Graph Data Structure

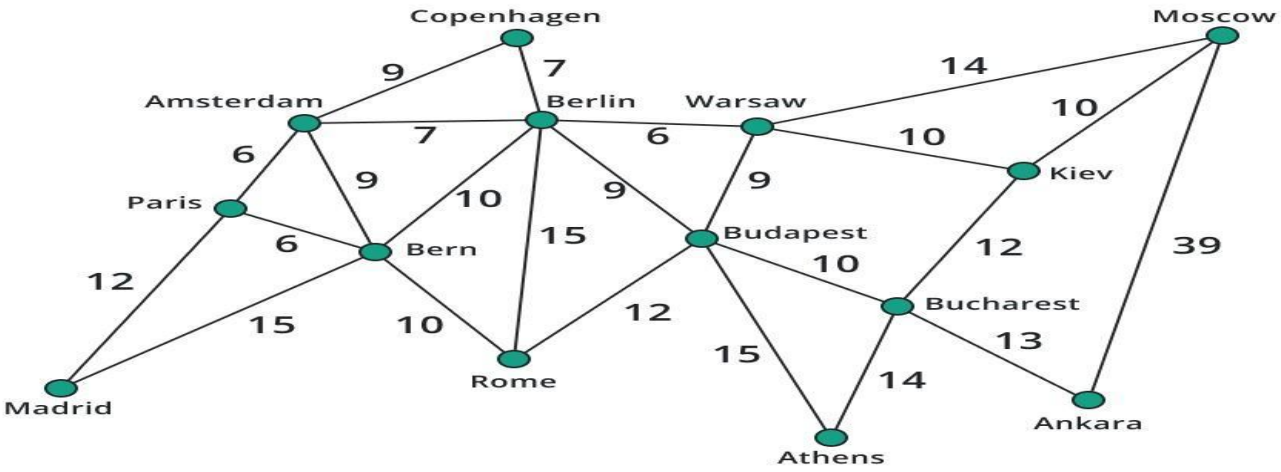
Introduction

A navigation system stores cities and their connections and calculates the shortest route between locations. It must represent road networks and their distances efficiently.

Suitable Data Structure: Graph

A graph consists of:

- Vertices: Represent cities.
- Edges: Represent roads connecting cities.
- Weights: Represent distance or travel cost.



Proposed Solution

Use a weighted graph with an adjacency list. Each city stores a list of connected cities and their road distances.

Example:

- City A to City B = 5 km
- City A to City C = 2 km
- City C to City B = 1 km
- City B to City D = 4 km

The shortest route from A to B is:

A → C → B = 2 + 1 = 3 km, instead of the direct 5 km route.

Suitable Algorithm: Dijkstra's Algorithm

Dijkstra's algorithm finds shortest paths from a source vertex when all edge weights are non-negative.

Steps:

1. Set the source city's distance to 0.
2. Set the distances of other cities to infinity.
3. Select the unvisited city with the smallest known distance.
4. Update the distances of its neighboring cities.
5. Repeat until the required destination is reached or all reachable cities are processed.

Time and Space Complexity

Let (V) be the number of cities and (E) the number of road connections.

Method	Time Complexity
Dijkstra with adjacency matrix	(O(V^2))
Dijkstra with adjacency list and binary heap	(O((V+E)log V))
Adjacency list storage	(O(V+E))

Advantages

- Represents complex road networks.
- Supports one-way and two-way connections.
- Stores distances as edge weights.
- Finds efficient routes between cities.
- Can be extended to use travel time or traffic cost as weights.

Conclusion

A weighted graph using an adjacency list, combined with Dijkstra's algorithm, is an appropriate choice for finding shortest routes efficiently in a navigation system.

3.ANS.

Web Browser Backward Navigation – Stack ADT

Introduction

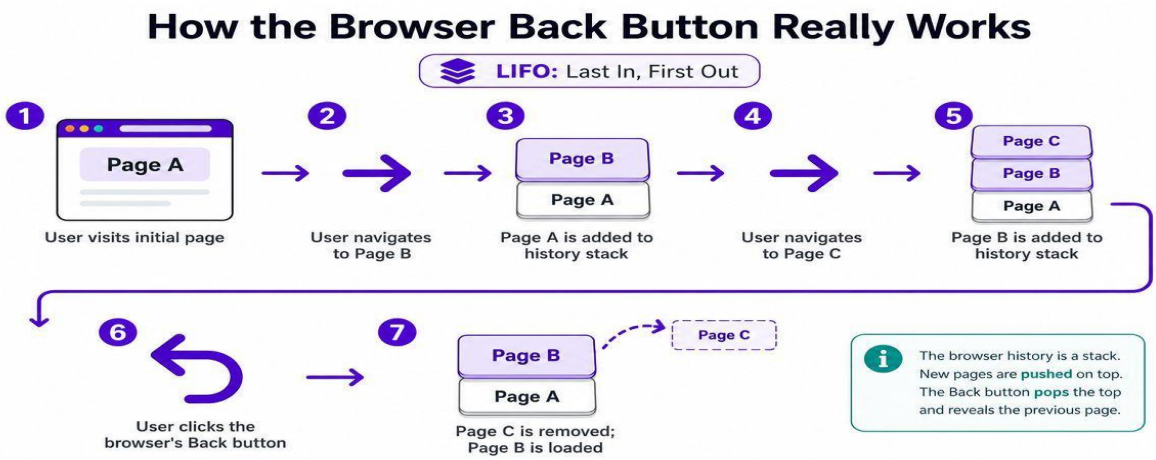
A web browser allows users to return to previously visited pages. A suitable data structure should store browsing history and support efficient backward navigation.

Suitable ADT: Stack

A stack follows the LIFO (Last In, First Out) principle. The last item added is the first item removed.

Stack operations:

- push() – adds a page to the history.
- pop() – removes and returns the most recent page.
- peek() – views the top page without removing it.
- Is Empty() – checks whether the stack is empty.



Working Example

Suppose the user visits:

Google → YouTube → Wikipedia

The stack contains:

Operation	Stack (bottom to top)
Visit Google	Google
Visit YouTube	Google, YouTube
Visit Wikipedia	Google, YouTube, Wikipedia
Press Back	Google, YouTube
Press Back again	Google

Each Back operation removes the current page from the history stack and returns to the previous page.

Time and Space Complexity

Operation	Complexity
Push a page	(O(1)) amortized
Pop a page	(O(1))
Peek at the top	(O(1))
Store \(n\) pages	(O(n))

Performance Efficiency

- Stack operations are fast.
- The most recently visited page is easy to retrieve.
- A dynamic stack can grow as browsing history increases.
- Backward navigation does not require searching the entire history.

Important: A real browser generally uses two stacks or an equivalent history mechanism to support both Back and Forward navigation. A single stack alone does not implement both directions.

Conclusion

The Stack ADT is suitable for browser backward navigation because its LIFO behavior naturally retrieves the most recently visited page, with constant-time push and pop operations.

4. ANS

Digital Library – Searching 50,000 ISBN Records

Introduction

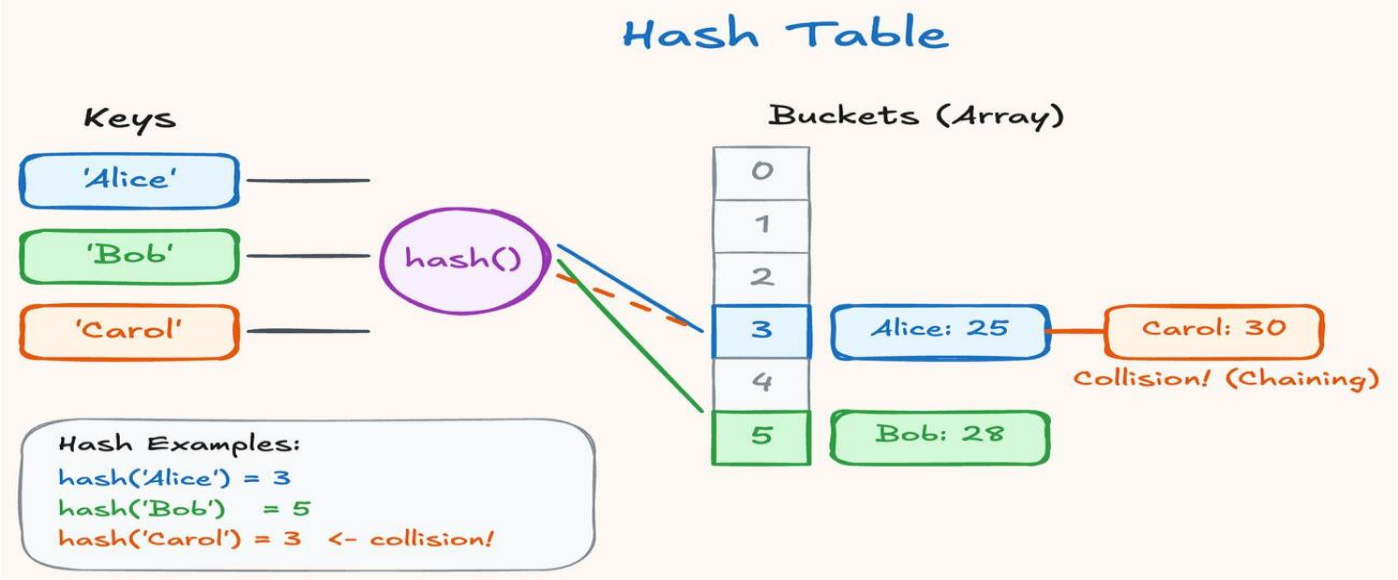
A digital library stores 50,000 book records. Users need to search for books using ISBN numbers quickly. Therefore, the selected data structure should minimize search time.

Suitable Data Structure: Hash Table

A hash table uses a hash function to convert an ISBN into an array index. The book record is stored at the corresponding position.

Example:

ISBN → Hash Function → Index → Book Record



Working Principle

1. The user enters an ISBN.
2. The hash function calculates an index.
3. The system accesses the corresponding location.
4. The ISBN is compared with the stored key.
5. If the key matches, the book details are returned.

If two ISBNs map to the same index, a collision occurs. Techniques such as separate chaining or open addressing handle collisions.

Time Complexity Comparison

Data Structure	Average Search	Worst-Case Search
Unsorted array	(O(n))	(O(n))
Sorted array using binary search	(O(log n))	(O(log n))
Balanced binary search tree	(Olog n))	(O(log n))
Hash table	(O(1))	(O(n))

For 50,000 records:

- Linear search may examine up to 50,000 records.
- Binary search needs approximately 16 comparisons in the worst case for a sorted array.
- A well-designed hash table typically finds a record in a small number of operations on average.

Space Complexity

A hash table storing (n) records requires (O(n)) space. Additional space may be needed for unused table capacity and collision handling.

Advantages

- Fast average ISBN lookup.
- Efficient for exact-match searches.
- Suitable for large collections of records.
- Supports insertion and deletion efficiently on average.

Limitations

- Collisions can reduce performance.
- Performance depends on the hash function and table load factor.
- Hash tables do not naturally maintain sorted ISBN order.
- Range searches and ordered traversal are better suited to tree-based structures.

Conclusion

A hash table is the most suitable choice when the primary requirement is fast exact-match searching by ISBN. Its average search complexity is \(O(1)\), making it efficient for a digital library containing 50,000 records.